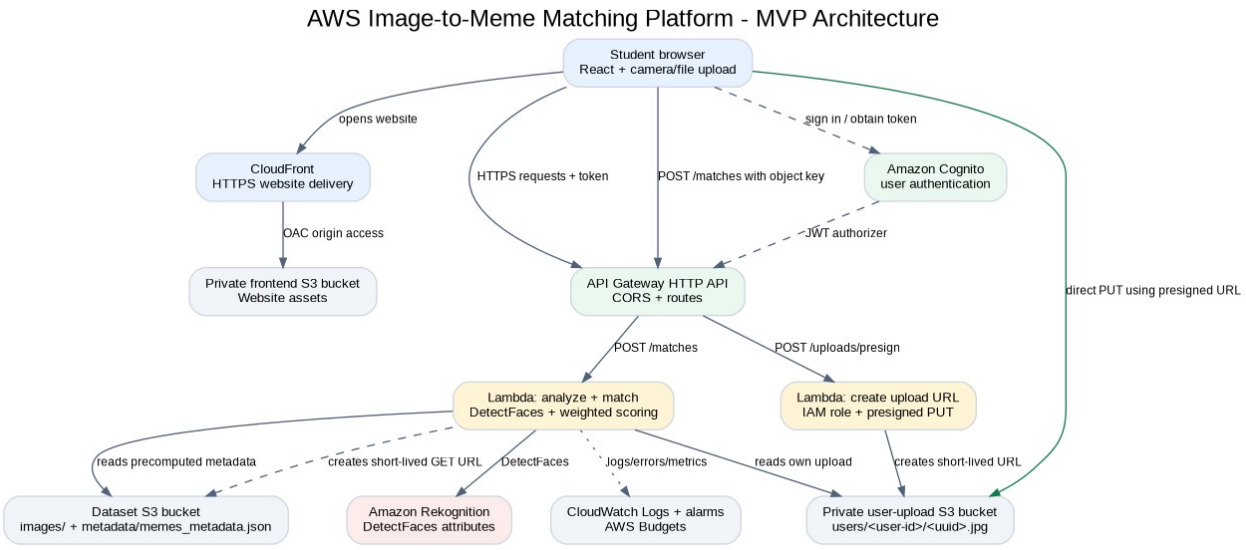
PROJECT IMPLEMENTATION GUIDE

AWS-Powered Image-to-Meme Matching Platform

An end-to-end build plan: dataset preprocessing, serverless backend, secure user uploads, website, testing, deployment and cost controls



Prepared for the project owner

Status at start of this guide: the dataset images have been uploaded to S3 and a metadata prefix has been created. The preprocessing and application pipeline are not yet confirmed as completed.

Version date: 26 October 2026 | Primary AWS Region: us-east-1 (N. Virginia)

How to use this guide

Follow the phases in order. Do not build the full public website before the dataset preprocessing test works. Each phase has an expected output and a checkpoint. The separate file preprocess_memes.py is a safe starter script that processes only 20 images by default; after reviewing that result, you can explicitly run it against the full dataset.

Key product decision

The initial MVP below matches facial expression and visible attributes (for example, smile, dominant detected emotion, mouth-open, glasses and pose). That is not the same as finding a face that genuinely resembles the student. A true look-alike feature needs a separate face-embedding similarity pipeline and should be a later phase.

Contents

- 1. Project summary and expected MVP
- 2. Current state and assumptions
- 3. Final architecture and data flow
- 4. AWS services and their responsibilities
- 5. S3 structure and dataset rules
- 6. Phase 0 - account, region and cost guardrails
- 7. Phase 1 - validate the dataset and permissions
- 8. Phase 2 - preprocess memes once
- 9. Phase 3 - validate metadata and define matching
- 10. Phase 4 - backend APIs and Lambda functions
- 11. Phase 5 - build the website
- 12. Phase 6 - authentication, privacy and isolation
- 13. Phase 7 - deploy the website on AWS
- 14. Phase 8 - test the complete system
- 15. Phase 9 - launch checklist and operations
- 16. Cost plan and credit protection
- 17. Common failures and fixes
- 18. Phase 2: actual facial resemblance / embeddings
- 19. Repository structure and deliverables
- 20. Presentation-ready project explanation
- Appendix A - exact CloudShell commands
- Appendix B - official AWS references

1. Project summary and expected MVP

The AWS-Powered Image-to-Meme Matching Platform is a web application for a college community. A student opens the site, captures or uploads a selfie, and receives a meme selected from a curated dataset of approximately 6,692 images (about 702 MB). The first version uses Amazon Rekognition to detect the student's face and facial attributes, then uses a weighted rule-based algorithm to rank preprocessed meme candidates.

Problem statement

Students have collections of memes but no interactive system that can quickly connect a user's captured photo with an appropriate face-expression or reaction meme. The proposed platform automates facial-attribute extraction, precomputes meme metadata once, and selects a relevant meme through a secure serverless web application. The objective is a lightweight, low-maintenance campus experience that demonstrates practical use of AWS storage, image analysis, APIs, serverless compute and access control.

MVP acceptance criteria

- ☐ A visitor can open a responsive HTTPS website and capture or choose a JPEG/PNG image.
- ☐ The image is uploaded directly to a private S3 upload bucket using a short-lived presigned URL; the browser never receives AWS access keys.
- ☐ The backend checks that the request belongs to the authenticated user and asks Rekognition to detect a face and attributes.
- ☐ The backend compares those attributes with precomputed meme metadata and returns a result from the eligible top-ranked memes.
- ☐ A user cannot list or retrieve another student's original uploaded image through the site or API.
- ☐ Uploaded selfies are deleted automatically after a short retention window, unless a clearly disclosed and justified feature requires longer retention.
- ☐ CloudWatch logs and a budget alert are configured; the project has been tested with no-face, multi-face, invalid-file and normal cases.

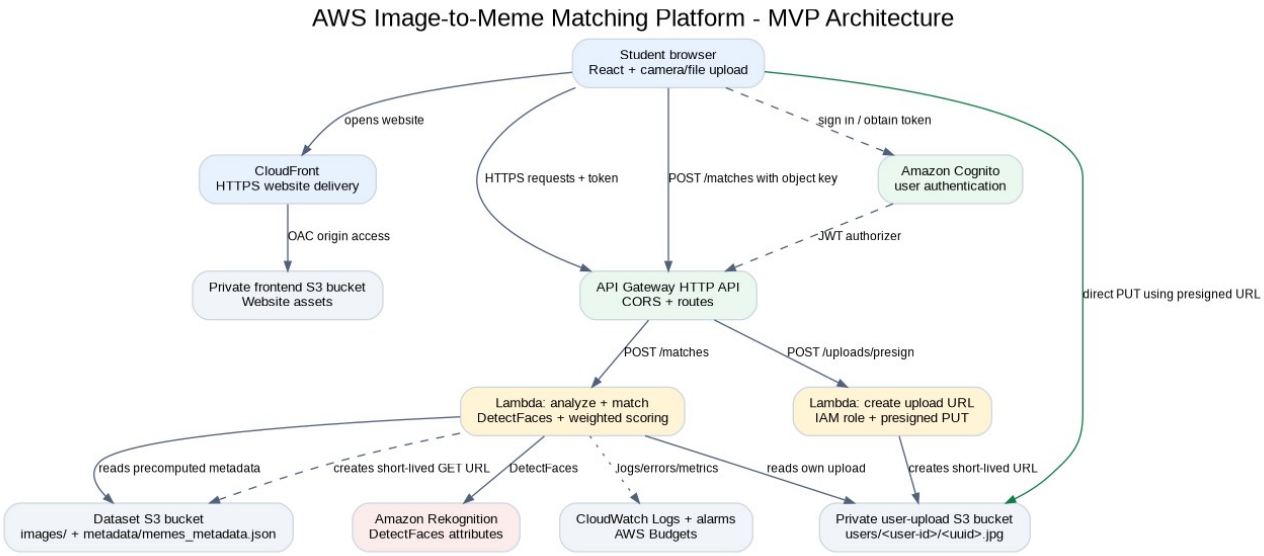
2. Current state and assumptions

Item	Known status	Next action
Dataset bucket	my-meme-database-2026	Keep the bucket; do not recreate it.
AWS region	get-bucket-location returned LocationConstraint: null; this denotes us-east-1 for a standard S3 bucket.	Use us-east-1 for Rekognition, Lambda and the initial APIs to avoid cross-region mismatch.
Dataset prefix	images/ contains files such as image_1.jpg and image_10.png.	Keep originals untouched; inventory formats and count.
Metadata prefix	metadata/ has been created, but it is not yet confirmed to contain generated analysis results.	The script will write metadata/memes_metadata.json.
Preprocessing	Not confirmed as run.	Test 20 items, inspect the results, then explicitly process all images.
Frontend and APIs	Not yet confirmed as built.	Build after preprocessing passes its validation checks.
Budget	You reported having about US$150 in AWS credits; the live usable balance has not been verified in this guide.	Check Billing/Credits in the AWS Console and create budget alerts.

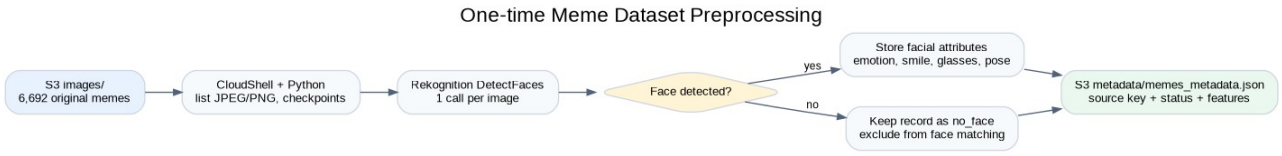
Important

The console's "folder" is a prefix in S3. Creating metadata/ does not itself classify images or run any service. The JSON file only exists after a script or application writes it.

3. Final architecture and data flow



A. One-time dataset flow



The existing meme dataset is analyzed once. For each supported JPEG/PNG object, the preprocessing script calls DetectFaces and stores its result under the object key in a JSON metadata file. Images with no detected face remain in the metadata as no_face records; the application excludes them from the expression-matching pool but does not delete the original images. Rekognition may miss faces in low-resolution, obscured, highly stylized or cartoon images, so "detected face" is an empirical result—not a guarantee that every visible face will be detected.

B. Each student request

1. The student signs in (Amazon Cognito for the privacy-preserving deployment).
2. The browser requests an upload URL from API Gateway. Lambda derives the user ID from the verified identity token and generates a random object key.
3. The browser uploads the selfie directly to the private user-upload S3 bucket with the short-lived presigned URL.
4. The browser calls POST /matches with the object key. The backend verifies that the key belongs to the authenticated user.
5. Lambda calls Rekognition DetectFaces on that one uploaded selfie and rejects the request if no usable face is detected.
6. Lambda loads the precomputed meme metadata, calculates candidate scores, randomly chooses among a small set of top-scoring candidates, and creates a short-lived URL to the selected meme.
7. The frontend displays the meme and a "Try again" action. The original selfie is deleted by lifecycle policy after the configured retention period.

4. AWS services and responsibilities

Service	Use in this project	MVP status
Amazon S3 - dataset bucket	Stores original memes and the precomputed metadata JSON. Existing bucket: my-meme-database-2026.	Already created
Amazon S3 - user-upload bucket	Private, separate storage for temporary student selfies. Enable Block Public Access.	Create
Amazon S3 - frontend bucket	Private origin for compiled website files; served through CloudFront Origin Access Control.	Create at deployment
Amazon Rekognition Image	DetectFaces returns face boxes, emotion estimates, smile, mouth-open, glasses, pose and other attributes.	Test, then use
AWS CloudShell + Python/boto3	Runs the one-time preprocessing script from the AWS Console using current console credentials.	Use now
AWS Lambda	Runs presigned-URL creation and per-user matching logic without an always-on server.	Create later
Amazon API Gateway HTTP API	Provides HTTPS endpoints and sends requests to Lambda.	Create later
Amazon Cognito User Pool	Authenticates students so user ownership can be checked by the backend.	Recommended before public launch
Amazon CloudFront	Delivers the website over HTTPS while the frontend S3 origin remains private.	Create at deployment
Amazon CloudWatch	Lambda logs, errors and metrics.	Enable with Lambda
AWS Budgets	Email alert when project costs reach configured thresholds; this is monitoring, not a hard spending cap.	Create now
DynamoDB (optional)	Can store user result history or a searchable per-meme index later. Not required for the first JSON-based prototype.	Defer unless needed

5. S3 structure and dataset rules

Existing dataset bucket

```
my-meme-database-2026/
├── images/
│   ├── image_1.jpg
│   ├── image_10.png
│   └── ... (the original meme dataset)
└── metadata/
    └── memes_metadata.json      # created by preprocessing
```

Recommended additional buckets

```
meme-user-uploads-<unique-suffix>/
└── uploads/<cognito-user-sub>/<uuid>.jpg   # private, temporary

meme-frontend-<unique-suffix>/
└── index.html, assets/...              # private S3 origin for CloudFront
```

- S3 bucket names must be globally unique. Add a short random suffix when creating the new buckets.
- Leave Block Public Access enabled for the dataset and user-upload buckets. Do not make user uploads publicly readable.
- S3 encrypts new objects at rest by default with SSE-S3. You do not need to enable KMS just to meet the MVP requirement; KMS introduces extra configuration and can add cost.
- Keep the dataset bucket and all compute/API resources in us-east-1 for the first implementation.
- Do not use the incoming user-upload prefix as a trigger for a Lambda that writes into the same triggering prefix. For this design, the /matches API invokes the processing Lambda directly; it avoids accidental recursive triggers.

6. Phase 0 - account, region and cost guardrails

1. Open the AWS Console and confirm the account and region selector. Select US East (N. Virginia), us-east-1, for this project's resources.
2. Open Billing and Cost Management → Credits and verify the remaining credit balance and expiry date. The approximate credit amount reported in chat is not proof of the live balance.
3. Open Billing and Cost Management → Budgets → Create budget. Create a monthly cost budget or expiring project budget, for example US$10–20, with actual and forecasted-cost email alerts at 50%, 80% and 100%. Choose an email you check.
4. Remember: AWS Budgets alerts do not automatically stop requests or prevent charges. Monitor Costs and Usage regularly, and delete or disable resources when the project is idle.
5. Do not create Rekognition face collections, Custom Labels training jobs, SageMaker endpoints, EC2 instances or NAT gateways for the first MVP. They are not needed for attribute-based matching.

7. Phase 1 - validate the dataset and permissions

Step 1. Confirm the S3 prefix and total object count

```
aws s3api get-bucket-location --bucket my-meme-database-2026
aws s3 ls s3://my-meme-database-2026/images/ --recursive --summarize
```

A null LocationConstraint means us-east-1 for this standard bucket. The second command prints a full listing and a total count/size at the end; it can take a little time because it lists every object.

Step 2. Validate image formats

The supplied starter script scans JPEG, JPG and PNG extensions. If the dataset also has GIF, WebP or other formats, leave those files in place but convert a copy to JPEG/PNG in a separate preprocessing step; do not silently assume Rekognition will accept every format.

Step 3. IAM permissions for preprocessing

Permission	Resource scope	Why
s3:ListBucket	arn:aws:s3:::my-meme-database-2026, restricted to images/ and metadata/ prefixes where possible	List dataset keys and inspect output

Permission	Resource scope	Why
s3:GetObject	arn:aws:s3:::my-meme-database-2026/images/* and metadata/memes_metadata.json	Read image keys and resume an existing metadata file
s3:PutObject	arn:aws:s3:::my-meme-database-2026/metadata/*	Write checkpoints and final metadata
rekognition:DetectFaces	*	Call DetectFaces; the action is not scoped to individual S3 object ARNs

CloudShell uses the AWS identity associated with your current Console session. If you see AccessDenied, fix the permissions for that identity or role; do not paste long-lived AWS access keys into a script or website.

Step 4. Test exactly one known face image first

```
python3 - <<'PY'
import boto3
bucket = "my-meme-database-2026"
region = "us-east-1"
key = "images/REPLACE_WITH_A_KNOWN_FACE_IMAGE.jpg"
rek = boto3.client("rekognition", region_name=region)
result = rek.detect_faces(
    Image={"S3Object": {"Bucket": bucket, "Name": key}},
    Attributes=["ALL"]
)
print("faces:", len(result.get("FaceDetails", [])))
for face in result.get("FaceDetails", []):
    print("smile:", face.get("Smile"))
    print("emotions:", face.get("Emotions"))
    print("glasses:", face.get("Eyeglasses"))
PY
```

Replace the placeholder with a real key for an image you have visually checked. A result of zero faces is not necessarily an API failure; the selected image may simply contain no face that Rekognition could detect. AccessDenied, InvalidS3ObjectException, or a region error must be fixed before batch processing.

8. Phase 2 - preprocess the meme dataset once

The starter script is provided separately as preprocess_memes.py. It is configured for the bucket and region above, calls DetectFaces only, writes checkpointed output to metadata/memes_metadata.json, skips records already completed, retries errors on a future run, and defaults to a safe 20-image test.

Exact procedure

1. Download preprocess_memes.py from the supplied file link in this conversation.
2. Open AWS Console → CloudShell in the same account and region. Use Actions → Upload file to upload preprocess_memes.py into the current home directory.
3. Run: python3 preprocess_memes.py. The default LIMIT=20 means it will attempt only 20 image objects.
4. Wait for the summary. Then open S3 → my-meme-database-2026 → metadata/ and verify that memes_metadata.json exists and has a non-zero size.
5. Download or inspect the JSON and review several records, including a face meme and an image that has no face. Confirm that face_count, faces, status and detected attributes make sense.
6. Only after the test is good, run: LIMIT=0 python3 preprocess_memes.py. The script resumes and skips already completed records, then processes the remaining supported images.
7. Wait for the final summary and download a local copy for inspection using the command in Appendix A. Do not run DetectLabels unless you specifically need object/scene tags; it is a separate API analysis and adds cost.

What the script creates

A record per image key, including status (success, no_face, or error), face_count, and per-face attributes such as dominant detected emotion, smile, mouth-open, glasses, pose, quality and bounding box. This is an index of model outputs—not a human-verified truth label.

Expected output shape

```
{
 "images/image_1.jpg": {
  "key": "images/image_1.jpg",
  "status": "success",
  "face_count": 1,
  "faces": [{
   "dominant_emotion": "HAPPY",
   "smile": {"value": true, "confidence": 96.8},
   "eyeglasses": {"value": false, "confidence": 99.1}
  }]
 },
 "images/image_2.jpg": {
  "key": "images/image_2.jpg",
  "status": "no_face",
  "face_count": 0,
  "faces": []
 }
}
```

The values above are an illustrative schema, not actual results from your S3 objects.

9. Phase 3 - validate metadata and define matching

First, measure dataset coverage

- Count total processed records, records with one or more faces, records with zero faces, and records with errors.
- Review at least 50–100 images across the dataset, not just the first 20, and compare the actual image with the detected attributes. Face detection can miss stylized/cartoon faces or detect only a subset of faces in a group.
- For a meme with multiple detected faces, retain all attributes in metadata but use the largest face box as the primary face for the initial matcher. This is a simple reproducible rule; a future UI can let admins choose the intended face.
- Do not delete “no_face” originals. Exclude them only from the first face-expression candidate pool; they can later be tagged using text or image concepts if desired.

Rule-based weighted scoring (MVP)

At runtime, Rekognition analyzes the student photo. The matching Lambda compares the student’s detected attributes with the primary detected face on each eligible meme. Use an explainable score rather than training a model for the first version.

Feature comparison	Example weight	How to score
Dominant emotion	0.50	1 if the top detected emotion matches, otherwise 0; optionally use confidence-weighted similarity.
Smile	0.25	1 if both records agree that a smile is present/absent; ignore low-confidence attributes.

Feature comparison	Example weight	How to score
Mouth open	0.10	1 if values agree, otherwise 0.
Pose similarity	0.10	Compare yaw/pitch differences after normalizing angle differences to 0–1.
Eyeglasses	0.05	1 if both values agree; treat missing/low-confidence values as unknown.

Example formula: score = 0.50 × emotion_match + 0.25 × smile_match + 0.10 × mouth_open_match + 0.10 × pose_similarity + 0.05 × glasses_match. Normalize over available features when some attributes are unknown. Apply a face-quality threshold before scoring, test weights on a labeled sample, and do not claim that these initial weights are objectively optimal.

Choose a result without always repeating the same meme

1. Filter to records with status=success and face_count > 0.
2. Compute a score for each eligible meme and sort descending.
3. Keep the top 3–5 candidates above a minimum score. If there are fewer candidates, fall back to a broader emotion-only pool.
4. Randomly choose from that small top pool; optionally exclude the last few meme keys shown to the same user/session.
5. Return only the meme image URL and non-sensitive match explanation. Do not return the uploaded selfie URL to the client unless needed for a short preview.

Do not confuse three different tasks

DetectFaces = finding faces and expression/attribute estimates. Face identity search = deciding whether a face is likely the same person. Look-alike similarity = ranking faces that visually resemble a user. The MVP supports the first task and a rule-based match, not a proven look-alike model.

10. Phase 4 - backend APIs and Lambda functions

Use two small Lambda functions for the first backend: one issues a presigned upload URL, and the other analyzes a user photo and matches a meme. API Gateway HTTP API routes requests to these functions. The frontend must never call Rekognition directly and must never contain AWS access keys.

API contract

Method + route	Input	Successful response
GET /health	No body; optional deployment test	{"status":"ok"}
POST /uploads/presign	{"contentType":"image/jpeg"}	{"uploadUrl":"...","objectKey":"uploads/<verified-user-sub>/<uuid>.jpg","expiresIn":300}
POST /matches	{"objectKey":"uploads/<verified-user-sub>/<uuid>.jpg"}	{"memeUrl":"...","score":0.91,"matchType":"expression-attributes"}

Lambda A - createUploadUrl

1. Read the verified Cognito subject (sub) from the authorizer claims; never accept a client-supplied userId as proof of ownership.
2. Validate that the request's content type is JPEG or PNG. Generate a random UUID file name and a key under uploads/<sub>/. Set a short presigned PUT expiry, for example five minutes.

3. Return the URL and object key. Do not expose the bucket's credentials or provide ListBucket permission to the student.

Lambda B - matchMeme

1. Require a valid authenticated request and verify the submitted object key starts with uploads/<the verified Cognito sub>/. Reject keys belonging to another user.
2. Use S3 HeadObject (or an equivalent controlled check) to validate that the object exists, its size is within the limit and its content type is allowed.
3. Call Rekognition DetectFaces against the uploaded S3 object. If no face is detected, return a clear "No face detected; try a clearer photo" response and do not produce a random irrelevant result.
4. Load metadata/memes_metadata.json from the dataset bucket. For a simple prototype, cache it in the Lambda execution environment after the first invocation; remember that Lambda can start a new environment, so caching is an optimization, not persistent storage.
5. Compute weighted scores, select from the top candidate pool and create a short-lived presigned GET URL for the chosen meme object.
6. Return JSON only. Log request IDs, durations, error codes and chosen meme key; do not log raw selfie bytes or full presigned URLs.

API Gateway setup

1. In AWS Console → API Gateway → Create API → HTTP API, add Lambda integrations for createUploadUrl and matchMeme.
2. Create routes: GET /health, POST /uploads/presign and POST /matches. Set a Cognito/JWT authorizer on upload and match routes before public deployment.
3. Configure CORS with the final CloudFront website origin only, allow POST/GET/OPTIONS as needed, and allow Content-Type and Authorization headers. Avoid wildcard origins in production.
4. Deploy the API to a stage, copy the invoke URL, and store it in the frontend environment as a public API base URL. It is not a secret; credentials are never stored in frontend code.

Use a separate Lambda execution role for each function and grant only the required S3 and Rekognition actions. Set practical memory/timeout values and review them in CloudWatch after initial tests. Lambda execution timeout is limited to 15 minutes, but a normal single-user match should be much shorter; the full dataset job belongs in the one-time script, not a single synchronous Lambda request.

11. Phase 5 - build the website

Recommended fast stack: React + Vite for the UI, plain CSS or a small component library for styling, Amazon Cognito for sign-in, and fetch() to call API Gateway. This keeps the site static so it can be delivered from S3 + CloudFront.

Screens and UI states

Screen/state	Must include
Landing page	Project name, concise explanation, "Start matching" action, privacy note.
Consent + upload	Clear notice that the image is processed by AWS facial-attribute analysis, permitted file types, max-size message and consent checkbox.
Camera/file input	Mobile capture option and file picker; preview before upload; remove/retake option.
Processing	Loading state such as "Analyzing facial expression…" and error handling if the request takes longer.
Result	Selected meme, "try another" action, brief match explanation, and a link to report an inappropriate result.

Screen/state	Must include
Privacy/About	What is stored, why, who can access it, retention window, contact/feedback and limitations.

Browser-to-AWS sequence

1. Get valid Cognito access/ID token.
2. POST {contentType} to /uploads/presign with Authorization: Bearer <token>.
3. PUT the file bytes directly to the returned presigned S3 URL.
4. POST {objectKey} to /matches with the same token.
5. Display the returned memeUrl; allow a retry or another match.
6. Show friendly errors for invalid file, no face, throttling, network failure and server error.

Build and run locally

```
npm create vite@latest meme-match-frontend -- --template react
cd meme-match-frontend
npm install
npm run dev

# Before deployment, set VITE_API_BASE_URL in .env.production.
npm run build
# Vite writes the deployable site to dist/
```

The native camera option can start with an input such as <input type="file" accept="image/*" capture="user">, which often opens the camera on mobile browsers. A desktop webcam preview using getUserMedia can be added as a later enhancement; always provide a file-picker fallback.

12. Phase 6 - authentication, privacy and isolation

Control	Implementation requirement
Authentication	Use Cognito before launching to the college. Restrict self-sign-up or use an invite/allowlist if it should be college-only.
User key ownership	Backend derives the user subject from the verified JWT and writes/accepts only keys under uploads/<sub>/. The frontend cannot pick another sub.
S3 privacy	Block public access on user and dataset buckets. Students get no ListBucket permission and never receive long-term AWS credentials.
Short-lived links	Use presigned PUT/GET URLs with a small expiration window. Treat a URL as a bearer capability: anyone who gets it can use it until it expires.
Retention	Create an S3 Lifecycle rule to expire uploads/ objects after a short window, such as 1 day. A lifecycle rule is a backup to application cleanup, not a replacement for a disclosed retention policy.
Consent	Explain processing in plain language before a selfie is uploaded; get affirmative consent and allow users to cancel.
Admin access	Admin access should come from controlled AWS IAM permissions and be logged/audited; do not make all uploads visible through a public “admin” page.

Control	Implementation requirement
Data minimization	Do not collect or classify skin tone, ethnicity or gender for this meme matcher. It is unnecessary for matching expression memes.
Logging	Avoid logging selfies, base64 payloads, Cognito tokens, personal identifiers beyond what is operationally necessary, and complete presigned URLs.

Privacy warning

A face photo is sensitive personal information in practical terms even if the prototype uses it only for entertainment. Publish the retention and access policy, restrict admin access, and confirm applicable college policies before collecting photos from 500–1,000 students.

13. Phase 7 - deploy the website on AWS

1. Create the private frontend S3 bucket with a unique name. Keep all Block Public Access settings enabled.
2. Build the website locally with npm run build and upload the contents of dist/ to the frontend bucket (not the dist directory itself).
3. Open CloudFront → Create distribution. Select the frontend bucket as a regular S3 origin, create/choose Origin Access Control (OAC), and choose to sign requests. Do not use the public S3 website endpoint as the OAC origin.
4. Allow the CloudFront distribution access to read the frontend bucket through the generated bucket policy; keep the bucket itself private.
5. Set Default root object to index.html. Enforce HTTPS/redirect HTTP to HTTPS. Verify that the CloudFront URL loads the home page.
6. Set the frontend's VITE_API_BASE_URL to the deployed API Gateway invoke URL and rebuild/re-upload when it changes.
7. Configure API Gateway CORS to allow the CloudFront URL. Then test from the actual deployed website, not only localhost.
8. If you later use a custom domain, use Route 53 or your DNS provider and an ACM certificate in the region CloudFront requires; this is optional for the first release.

14. Phase 8 - test the complete system

Test	Expected outcome	Pass?
Known clear-face selfie	Upload succeeds; one face is returned; a matching meme displays.	☐
No-face image	Friendly retake message; no incorrect "random" result.	☐
Multiple faces in selfie	Either prompt for a single person or consistently select the largest face; behavior is documented.	☐
Unsupported file / too-large file	Rejected by frontend and backend without invoking Rekognition.	☐
Expired presigned URL	Upload fails; frontend requests a new URL.	☐
User A requests User B's object key	Backend returns 403/404; no URL or image bytes are returned.	☐
Metadata includes no-face record	That record is excluded from the face-matching candidate pool.	☐

Test	Expected outcome	Pass?
Metadata missing / malformed	Backend returns a controlled error and logs it; no exception stack leaks to the browser.	☐
CORS from deployed site	Browser request succeeds only from intended allowed origin(s).	☐
Retention	Test object is expired/deleted according to configured policy; confirm lifecycle rules are active.	☐
Cost/monitoring	CloudWatch logs are present, budget alerts are configured, and no unused test resource remains.	☐

Minimum pilot before 500–1,000 students

- Test with 5–10 consenting people, then 25–50, and only then expand to the larger college group.
- Measure average match latency, error rate, detected-face rate, how often results feel relevant, and cost per 100 requests.
- Add API throttling/rate limits and maximum file size. A public endpoint with no abuse control can consume credits quickly.
- Make the project owner responsible for removing inappropriate memes and handling removal requests.

15. Phase 9 - launch checklist and operations

- ☐ Dataset preprocessing completed; metadata count equals the number of supported images scanned (including no-face and error records).
- ☐ At least 50–100 dataset samples reviewed manually; a random sample from the entire dataset has been checked.
- ☐ One user's photo cannot be retrieved by another user through any API route.
- ☐ Only the frontend bucket is served by CloudFront; dataset and upload buckets remain private.
- ☐ Cognito, API Gateway, Lambda roles and S3 policies use least privilege.
- ☐ CloudWatch log groups and useful error metrics are available; sensitive image data/tokens are not logged.
- ☐ AWS Budgets alerts are configured and the credit expiry date is known.
- ☐ S3 lifecycle expiry is active for student uploads; a clear privacy notice is published.
- ☐ No unused custom-label training job, face collection, EC2 instance or other paid resource remains running.
- ☐ The frontend has camera/file-upload fallback, accessible errors, responsive design and a working contact/feedback route.

16. Cost plan and credit protection

The 702 MB dataset occupying S3 is not the main cost driver. The cost to watch is the number of analysis calls and any accidentally running compute resources. AWS's current Rekognition pricing page gives a standard Group 2 example rate of US$0.001 per analyzed image in the first tier; DetectFaces and DetectLabels are Group 2 APIs. AWS also states that calls to multiple APIs for the same image count as separate image analyses. Actual charges depend on region, usage tiers, eligibility and the current pricing page.

Workload	Indicative planning estimate before eligible credits/free tier	Notes
One DetectFaces call for each of 6,692 memes	About US$6.69 using the published example rate	Dataset preprocessing is one-time; script checkpoints are intended to avoid duplicate successful calls.

Workload	Indicative planning estimate before eligible credits/free tier	Notes
Also call DetectLabels for all 6,692 memes	About another US$6.69 at the same example rate	Optional. Do not pay for labels unless you need object/scene tags.
1,000 student uploads, one DetectFaces call each	About US$1 using the same example rate	Excludes uploads, API calls, other AWS services and regional/tier differences.
702 MB S3 dataset storage	Very low monthly storage charge under typical S3 Standard rates	Requests and data transfer are billed separately; inspect Billing for actual charges.

- Do the first 20-image run, inspect output, then process all. If it fails, fix the cause before retrying the full collection.
- Call DetectFaces only for the first version. Add DetectLabels only if scene/object metadata helps your algorithm.
- Do not call Rekognition against the full meme dataset on each student request. Precompute once; each student request should analyze only that student's image and compare with stored metadata.
- Do not assume a credit balance is a hard cap. Alerts may arrive after usage; check Cost Explorer/Billing and remove resources you no longer need.
- AWS's listed eligible free-tier allowances and credits vary with account creation date and plan; verify your own account's billing/credit page instead of subtracting an assumed amount.

17. Common failures and fixes

Symptom	Likely cause	Action
LocationConstraint is null	Standard S3 bucket is in us-east-1.	Use region us-east-1 for this project.
Broken pipe after `... | head`	head exits after a few lines while aws writes more output.	Harmless; use full summary or redirect output to a file.
AccessDenied calling DetectFaces	Current CloudShell IAM identity lacks rekognition:DetectFaces.	Fix the current identity's least-privilege permissions.
InvalidS3ObjectException	Wrong bucket/key, object unreadable, or wrong region.	Check exact case-sensitive key, access and region.
No faces found	Image has no detectable face, face is too small/obscured/stylized, or the chosen file is not a person image.	Test a known clear-face meme; retain no_face records and measure coverage.
Throttling / ProvisionedThroughputExceeded	Requests too fast or service quota reached.	Use the supplied retry-capable script, lower request rate and check service quotas.
JSON not found in S3	Preprocessing has not run, output prefix differs, or run failed before checkpoint.	Check CloudShell output and the exact metadata object key.
CORS error in browser	Allowed origin/method/header configuration does not match deployed website.	Update HTTP API CORS and redeploy if required.
Users can access another image	Backend trusts client-supplied keys or objects are public.	Stop public access, validate Cognito subject against object key on every request, rotate/revoke exposed URLs as needed.
Cost jumps unexpectedly	Duplicate analysis, retry loop, public API abuse, or running resources.	Check Cost Explorer by service, CloudWatch invocation counts, API rate limits and resource inventory.

18. Phase 2: actual facial resemblance / embeddings

If the product requirement means “choose a meme whose face visually resembles the student,” expression attributes alone are insufficient. A happy student and a smiling meme might match expressions but have very different facial structures.

A separate look-alike pipeline

1. Select a face-embedding model appropriate to the source image style and review its license and model card. Real-photo face models may perform poorly on cartoon or heavily stylized meme faces.
2. Run face crop extraction on each meme that has a valid face bounding box, then compute and save one embedding per chosen face in a one-time batch job.
3. For each student upload, crop the detected face, compute its embedding with the same model/version, and compare it with stored vectors using cosine similarity or another validated metric.
4. Return top-ranked meme faces, optionally re-rank with expression attributes, then randomly choose from the top few.
5. Benchmark on a consented, representative sample and check whether the results actually appear similar to people. Embedding similarity is not a guarantee of identity or social meaning.
6. Treat embeddings and face crops as sensitive data. Apply consent, access control, deletion and model/version tracking. Do not silently create a face-identification database of students.

Recommended sequencing

Finish the expression-based MVP first. Once upload, permissions, metadata, matching, website and retention work end-to-end, add embeddings as a separately tested feature. Do not try to build both inference pipelines while the first upload/API flow is still untested.

19. Repository structure and deliverables

```
aws-meme-matcher/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── api/client.js
│   │   └── App.jsx
│   ├── public/
│   ├── .env.example
│   └── package.json
├── backend/
│   ├── create_upload_url/handler.py
│   ├── match_meme/handler.py
│   └── shared/matching.py
├── scripts/
│   └── preprocess_memes.py
├── docs/
│   ├── architecture.png
│   └── project-report.md
├── tests/
│   └── test_matching.py
├── .gitignore
└── README.md
```

- Do not commit private student photos, the entire 772 MB meme dataset unless its license permits redistribution, Cognito secrets, AWS access keys, or production environment files.
- Keep generated metadata versioned or backed up in S3; record the preprocessing date, API used, script version and dataset count.

- README should document architecture, setup, deployment, API contracts, limitations, data retention and cost assumptions.

20. Presentation-ready project explanation

30-second summary

Suggested pitch

"We are building a serverless AWS-powered Image-to-Meme Matching Platform for college students. A student uploads a selfie through a website. The image is stored privately in S3, analyzed by Amazon Rekognition for visible facial attributes, and matched against a preprocessed catalogue of memes. The catalogue is analyzed only once, so each request processes the student image rather than re-analyzing thousands of memes. API Gateway and Lambda connect the website to the backend, while Cognito, IAM and short-lived S3 URLs protect uploads and results."

Questions you should be ready to answer

Question	Strong answer
Why AWS serverless?	S3 provides object storage; Rekognition provides managed image analysis; Lambda runs on demand; API Gateway exposes HTTPS routes. There is no always-on web server to manage for this MVP.
Why precompute meme metadata?	It avoids thousands of API calls for every student request. Dataset analysis happens once, while each request analyzes only the new selfie.
What algorithm is used?	The initial matcher is a weighted rule-based scoring algorithm over detected emotion, smile, mouth-open, pose and glasses, then random selection from the highest-ranked candidates.
Is this face recognition?	The MVP uses face detection and attribute analysis, not person identification. It does not prove that a meme face is the same person or a true look-alike.
What if a meme has no detectable face?	The record stays in metadata as no_face and is excluded from the initial face matcher. Rekognition can miss stylized or low-quality faces, so sample validation is required.
How do you protect users?	Private S3 buckets, Cognito authentication, per-user object keys, backend ownership validation, no browser AWS keys, short-lived presigned URLs, logging controls and automatic upload expiry.
How do you control cost?	One-time preprocessing, DetectFaces only at first, small image-size limit, budget alerts, rate limits, no always-on compute and no custom model training for the MVP.
What is the next improvement?	A separately tested face-embedding index for look-alike similarity, plus better meme content tags and user feedback to rank results.

Appendix A - exact CloudShell commands

Use these commands in AWS CloudShell after selecting the same AWS account. The script file supplied separately should be uploaded into CloudShell using Actions → Upload file.

A1. Check bucket region and files

```
aws s3api get-bucket-location --bucket my-meme-database-2026
aws s3 ls s3://my-meme-database-2026/images/ --recursive --summarize
```

A2. Run the safe 20-image preprocessing test

```
python3 --version
python3 preprocess_memes.py
```

A3. After reviewing the test output, process all supported images

```
LIMIT=0 python3 preprocess_memes.py
```

The script checkpoints its JSON output to s3://my-meme-database-2026/metadata/memes_metadata.json, so completed records can be skipped on a rerun. Error records are attempted again on a later run. Check the script output and metadata status counts before proceeding.

A4. Download metadata to inspect counts

```
aws s3 cp s3://my-meme-database-2026/metadata/memes_metadata.json ./memes_metadata.json
python3 - <<'PY'
import json
from collections import Counter
with open("memes_metadata.json", encoding="utf-8") as f:
    data = json.load(f)
print("records:", len(data))
print("statuses:", dict(Counter(v.get("status") for v in data.values())))
print("faces detected in images:", sum(1 for v in data.values() if v.get("status") == "success"))
print("no-face images:", sum(1 for v in data.values() if v.get("status") == "no_face"))
print("error records:", sum(1 for v in data.values() if v.get("status") == "error"))
PY
```

Do not continue to API/frontend implementation until the JSON contains sensible records, the face/no-face counts are understood, and error records have been investigated.

Appendix B - official AWS references

Pricing, free-tier rules and service features can change. These official pages were checked for this guide on 26 October 2026; recheck them before a public launch.

- Amazon Rekognition pricing - API groups, per-image pricing, free tier and credits
- Amazon Rekognition DetectFaces - supported face attributes and S3 image input
- Amazon S3 default encryption - SSE-S3 enabled by default for new objects
- S3 presigned URL upload/download - temporary access without AWS credentials in the browser
- API Gateway HTTP APIs - integration with Lambda, CORS and auth options
- S3 event notifications with Lambda - asynchronous invocation and recursive-trigger warning
- CloudFront private S3 origins and Origin Access Control
- Amazon Cognito user pools - web/mobile user authentication
- AWS Budgets - create a cost budget and alerts
- Lambda execution timeout - maximum 900 seconds / 15 minutes